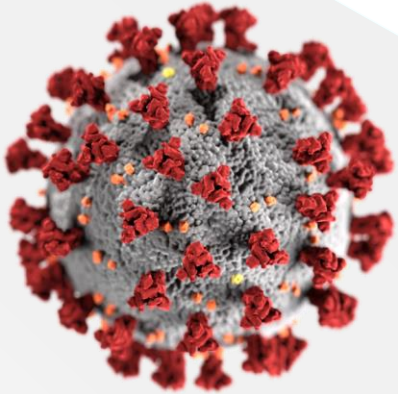
User Defined Data Types: Structures, Unions

CSE 1112
Lec Raiyan,
Dept of CSE, UIU,
raiyan@cse.uiu.ac.bd

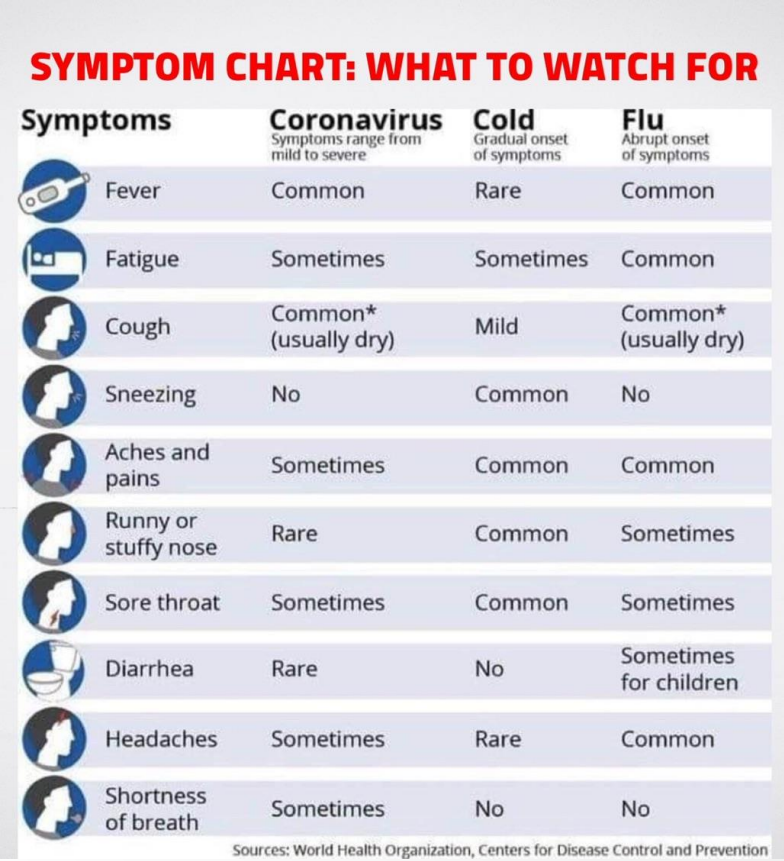
Let's make a covid symptom checker with what we learnt so far!



www.coronastate.mist.ac.bd

CSE 1112
Lec Raiyan, Dept of CSE, UIU, raiyan@cse.uiu.ac.bd

Solve with C



SYMPTOM CHART: WHAT TO WATCH FOR

Symptoms	Coronavirus Symptoms range from mild to severe	Cold Gradual onset of symptoms	Flu Abrupt onset of symptoms
Fever	Common	Rare	Common
Fatigue	Sometimes	Sometimes	Common
Cough	Common* (usually dry)	Mild	Common* (usually dry)
Sneezing	No	Common	No
Aches and pains	Sometimes	Common	Common
Runny or stuffy nose	Rare	Common	Sometimes
Sore throat	Sometimes	Common	Sometimes
Diarrhea	Rare	No	Sometimes for children
Headaches	Sometimes	Rare	Common
Shortness of breath	Sometimes	No	No

Sources: World Health Organization, Centers for Disease Control and Prevention

We've to find a way to represent this states in numerically!

What if we did -

Common = +1
Sometimes = + 0.5
Rare = - 0.5
No = -1

The user will select "Y/N" for each symptom and we'll map accordingly.

Max possible outcome can be +5 *

So, a +5 score = 100% probability of COVID-19

*For shortness, of breath gave a +1 score although it's "sometimes"

Let’s do it!

For solve, check corona.c

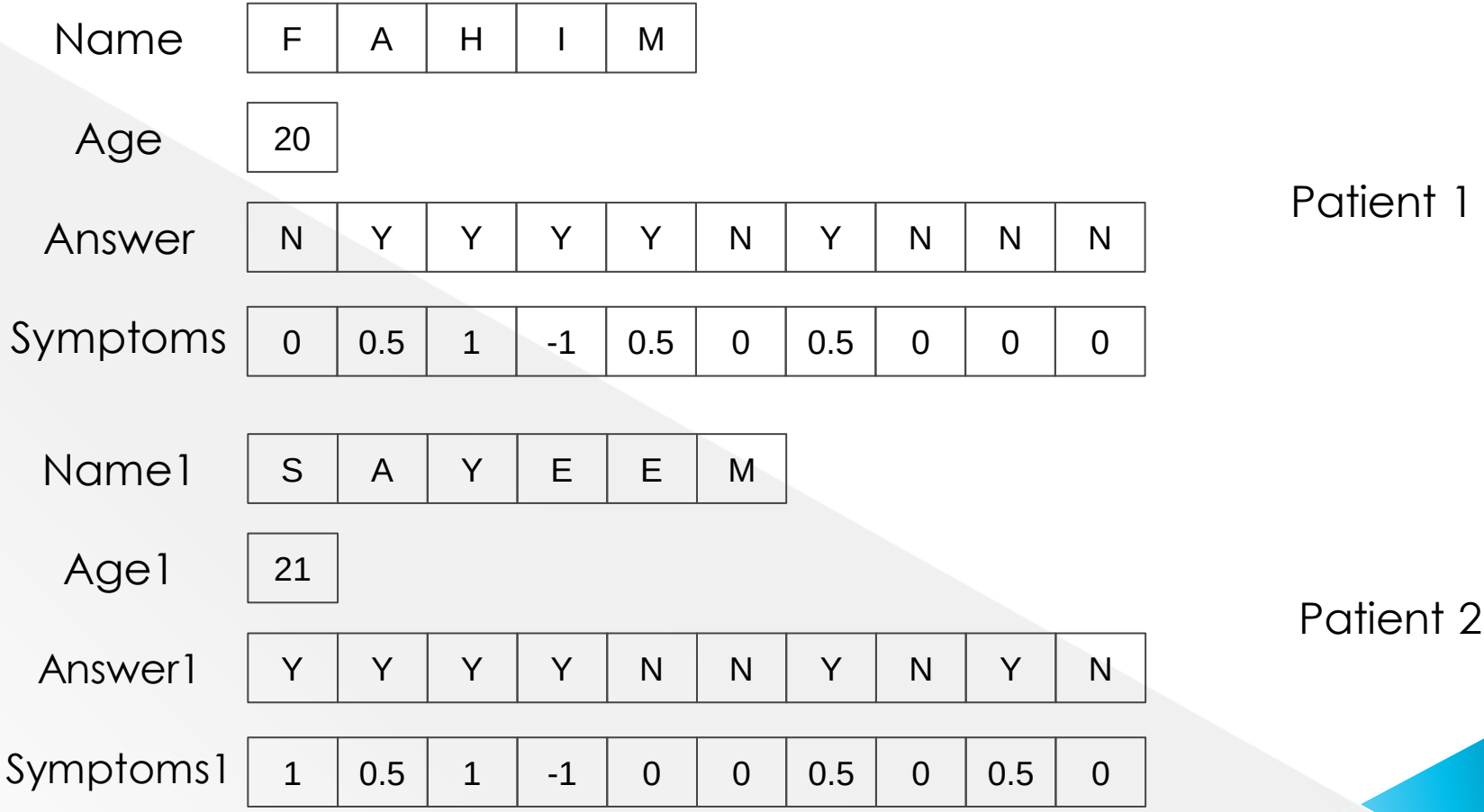
But what if we wanted store all the patients' data (name, age etc.) as a group?

And we wanted to store all data for multiple patients at the same time?

Name	F	A	H	I	M					
Age	20									
Answer	N	Y	Y	Y	Y	N	Y	N	N	N
Symptoms	0	0.5	1	-1	0.5	0	0.5	0	0	0

Patient 1

We'd have to declare 4 variables.



What of we had 2 patients? We'd have to declare 8 variables.

- What if we had more? Do we keep on declaring variables separately each time? No!
- Also, are these vars separate? Don't they not represent a patient's data in my system combinedly?
- If so, should they not be "grouped" together?
- *Also, don't I need these 4 vars each time I have a new patient in the system?
- So, each time I have a new patient in the system, should these 4 vars not be declared as a group automatically?

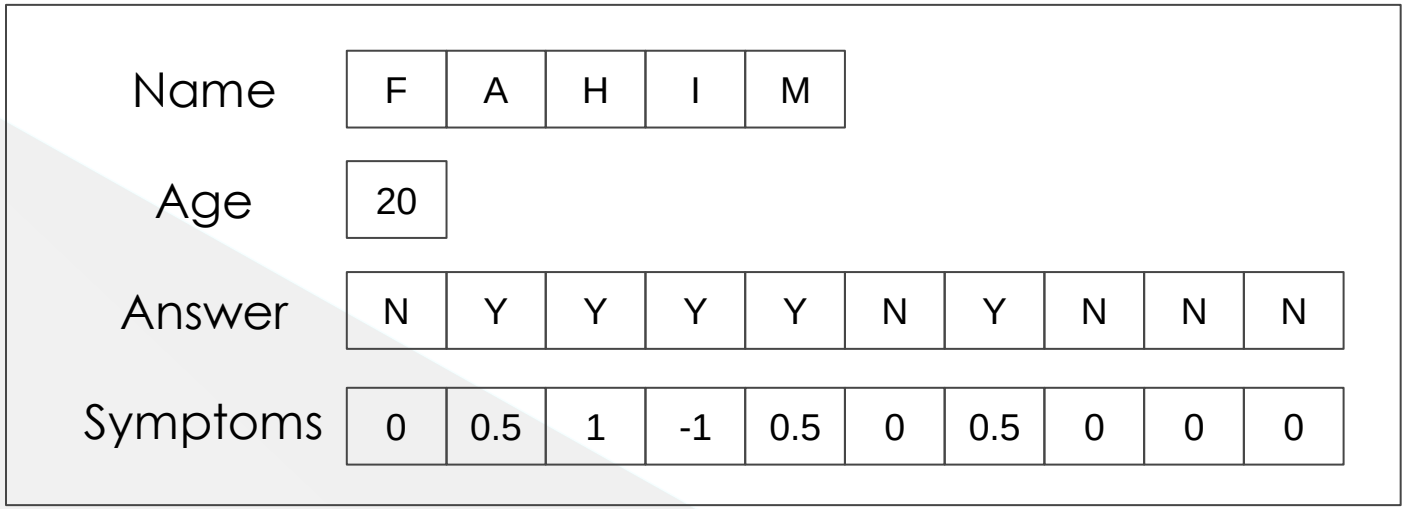
Answer is yes, yes, yes and yes!

* That is if we want each patient's data to remain, re-writing the same vars would erase the previous data.

- So, let's group them together. And each time a new patient enters, we'll declare a variable of that "group".
- And we'll do so by creating a composite new data type of our own! That composite new data type will have name (a string) + age (an int) + answers (a char array) + symptoms (a float array).
- Let's call this new data type "patients".
- This is a user-defined data type.

User Defined Data Types: Structures, Unions

patient
(a user defined new data type)



VS

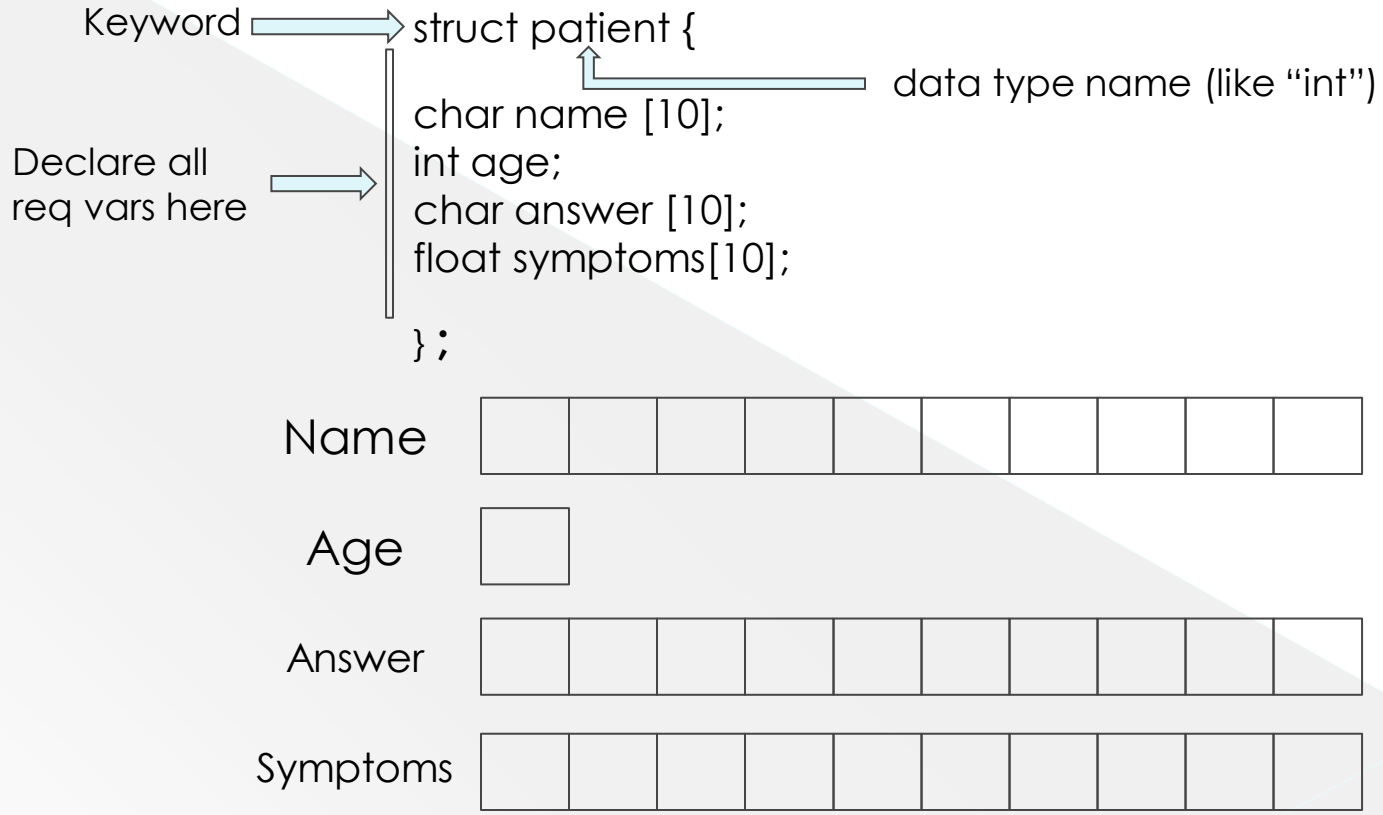
int
(a built-in data type)

a 25

Solution: User Defined Data Types

- So, we'll create new data types acc to our program's need.
- They can combination of "built-in" and even other user-defined data types!
- Once we declare them we can use them anywhere in the program after that.
- The main idea is, we'll group the attributes or data of a particular entity together. In this case "patients".
- These new data types are called user-defined data types.
- Declaring a variable of type "patient" will create allot enough memory space to hold all 4 variables.

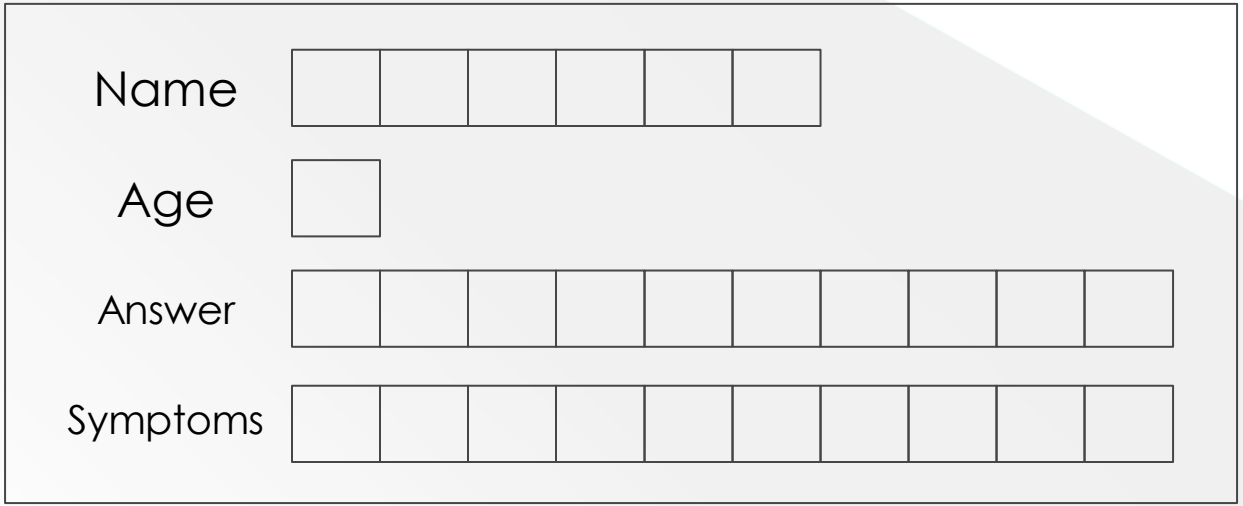
User Defined Data Type: Defining the Data Type



User Defined Data Type: Declaration

```
int main()
{
        struct patient p1;

}
```

Keyword Data type Variable name

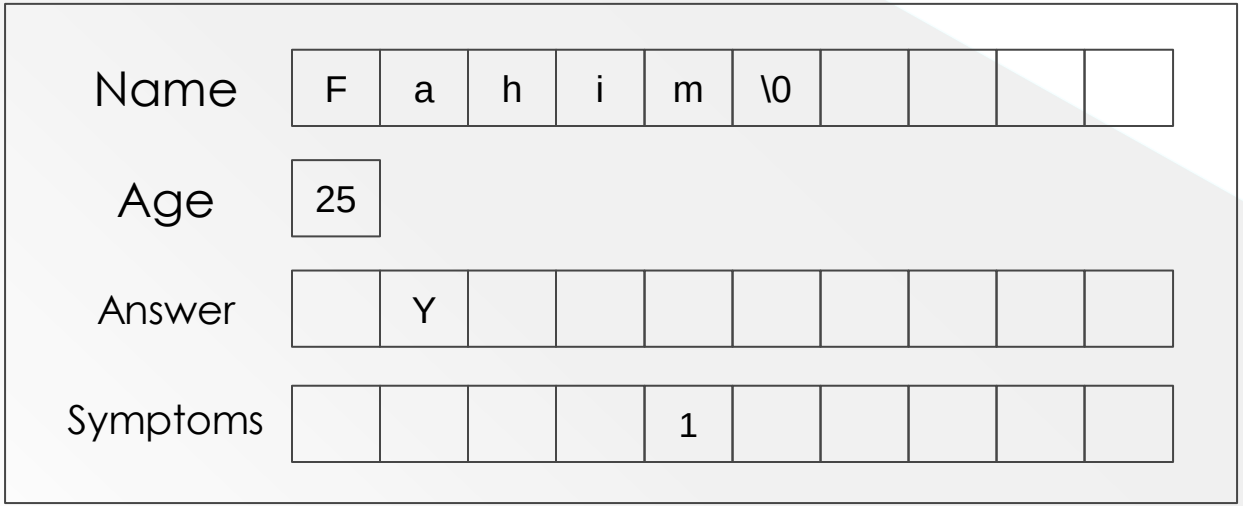



```
int main()
{
        int a;

}
```

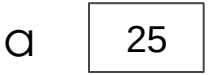
a

User Defined Data Type: putting value/taking input

```
int main()
{
        struct patient  p1;
        p1.age=25;
        gets(p1.name);  //say user gives input 'fahim'
        p1.answer[1]='Y';
        p1.symptoms[4]=1;
}
```

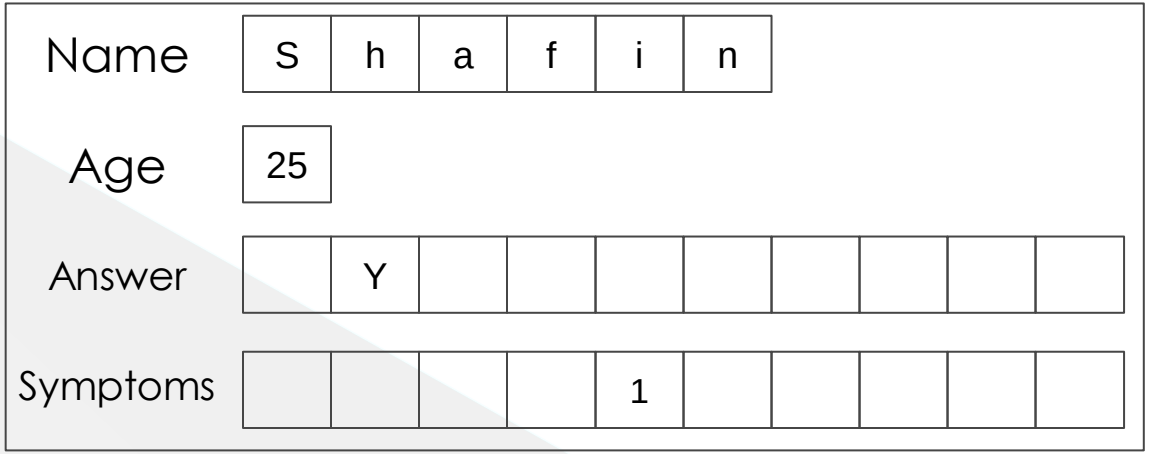



```
int main()
{
        int a;
        a=25;

}
```



The Entire Code

```
struct patient {

char name [10];
int age;
char answer [10];
float symptoms[10]

};


int main()
{
        struct patient p1;
        p1.age=25;
        gets(p1.name);  //say user gives input 'shafin'
        p1.answer[1]='Y';
        p1.symptoms[4]=1;
}
```



So, why do we need structure?

Arrays require that all elements be of the same data type. Many times it is necessary to group information of different data types.

Both C and C++ support data structures that can store combinations of character, integer floating point and enumerated type data. They are called a *structs*.

Structure

- A struct is a derived data type composed of members that are each fundamental or derived data types.

- A single struct would store the data for one object. An array of struct would store the data for several objects.

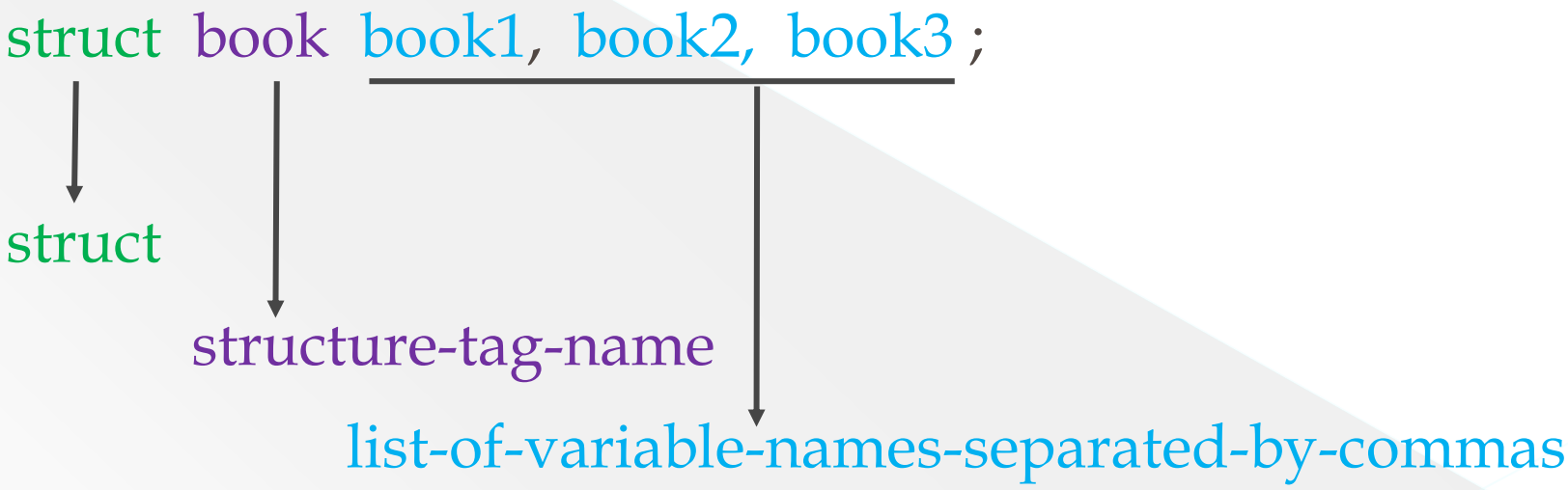
Defining Structure - another example

Declares a structure

Structure tag
Can be used to declare variables that have the tag's structure

```
struct book
{
  char  title[20];
  char author[15];
  int pages;
  float price;
};
```

Declaring Structure



Initializing Structure

```
struct book
{
   char  title[20];
   char  author[15];
   int pages;
   float price;
};
int main( )
{
 struct book book1 = {“ANSI C”, “E Balaguruswamy”, 200, 120.50};
}
```

Accessing Structure Members

Individual members of a *struct* variable may be accessed using the structure member operator. (the dot operator or the period operator ".":

book1.price ; *representing the price of book1*

Operations on Structure

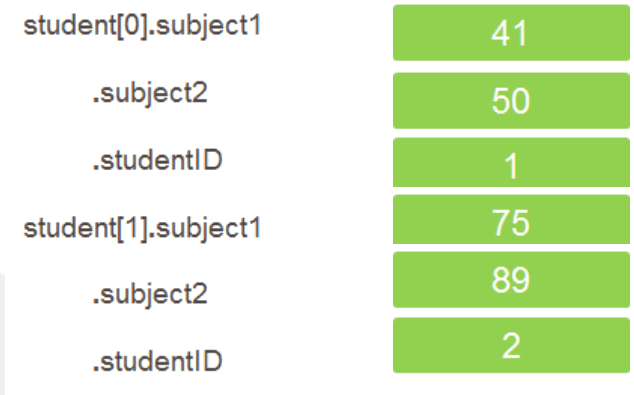
- strcpy(book1.title, “BASIC”);
- book1.pages = 250; [*assign values*]
- scanf(“%d”,&book1.pages); [*user input*]
- book1==book2; [*logical operations not allowed on the whole structure*]
- book1.pages!=book2.pages; [*logical operations are allowed on individual member with the dot operator along with its structure variable*]

Let’s re-do our program with structures!

For solve, check corona_with_structures.c

Array of Structure

```
struct marks
{
  int subject1;
  int subject2;
  int studentID;
} ;
int main( )
{
  struct marks student[2] = {{41,50,1} , {75, 89,2}};
}
```



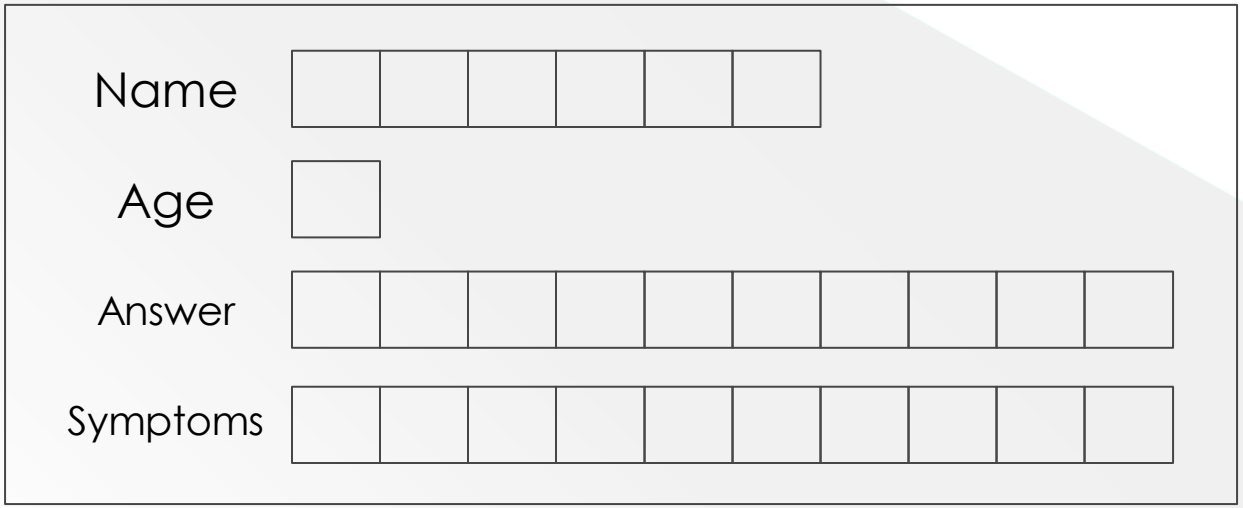
Let’s re-do again with array of structures!

For solve, check corona_with_array_of_structures.c

Structures with pointers

```
int main()
{
        struct patient * p1;

}
```

Keyword Data type Variable name (ptr)

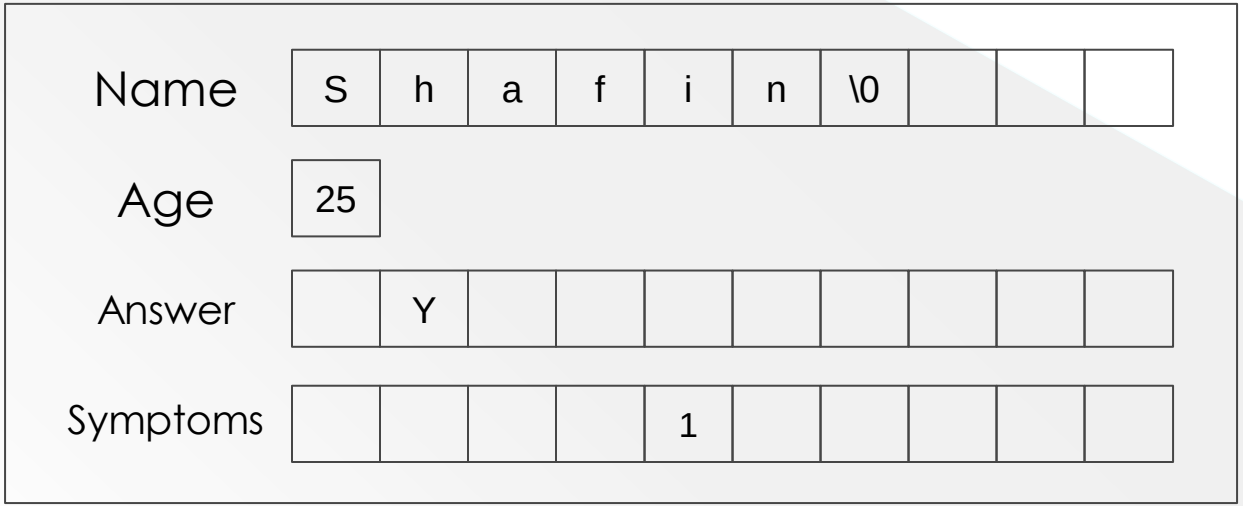



```
int main()
{
        int * a;

}
```

a

User Defined Data Type: putting value/taking input

```
int main()
{
        struct patient  p1;
        struct patient *ptr;
        ptr=&p1;
        ptr->age=25;  \\only difference

};
```



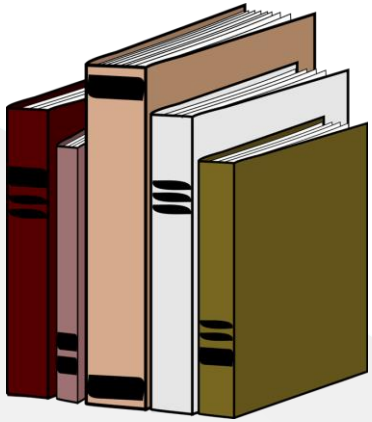

```
int main()
{
        int * a;
        int b;
        a=&b;
        *a=25;

}
```

Let's Solve Another Problem!

Library Management System, where users can:

1. Add new books to library
2. Check book info at any time
3. Search any particular book
4. Check out a book to a library member
5. Check a book back in.

So, a simple but fully working library automation software in C!



Let’s get to coding!

For solve, check library.c

Nested Structures

For example, the library book can be checked out to a library card holder (user). While checking out, we want to store the data. So, let us modify the library structure in such a way that it can now also hold the data of the person who checks a book out. We'll add another struct named 'person'.

- The keyword "struct" must be written before the variable name.
- Union studentID must be declared above struct marks.

```
struct person {
   char name[30];
    char address[150];
    char email[40];
    double contact_no;
};
```

```
struct library
{
  ...other vars of struct library
  struct person p;
} ;
```

Let’s re-do our program with nested structures!

For solve, check library_with_nested_structures and union.c

References

- https://www.programiz.com/c-programming/c-structures

Problems with Library.c?

Book names or customer/library member name might not be unique!

Solve: We should have an unique ID instead.

But, what if… The ID was more than one type?

Solve… ?

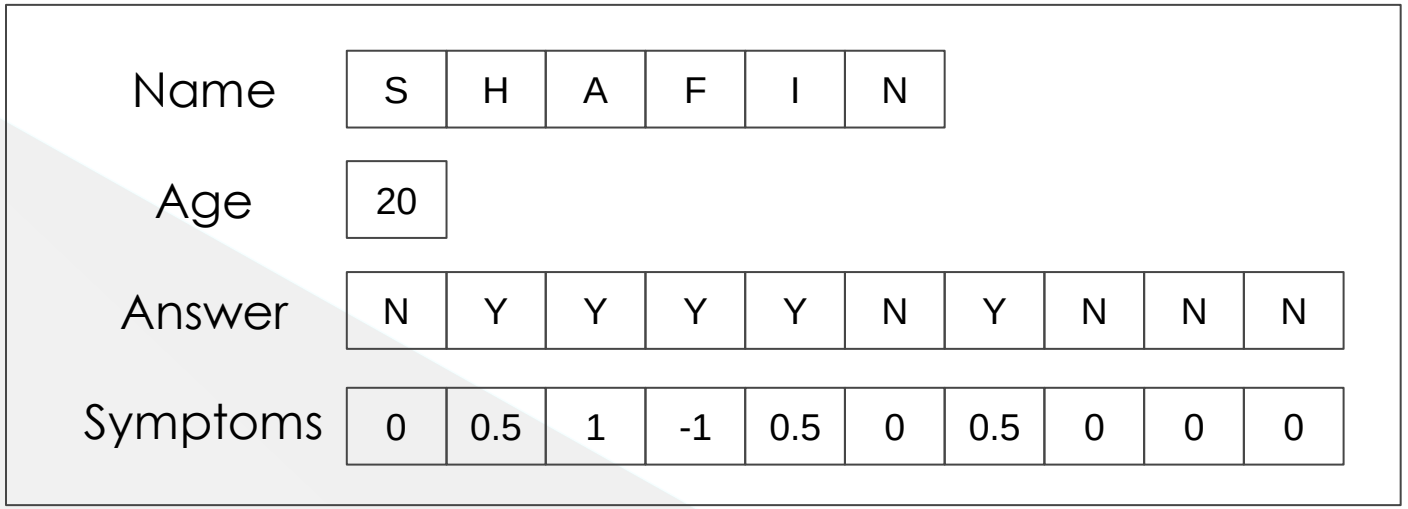
Check out library_with_nested_structures and union.c

Unions

- Another User defined Data Type.
- Can be a group of any built-in data types.
- However, only allocated enough space to store the **largest member** of that group!
- So what happens when you try to assign multiple vars of union together? Only the larger one is stored and other one is “corrupted”

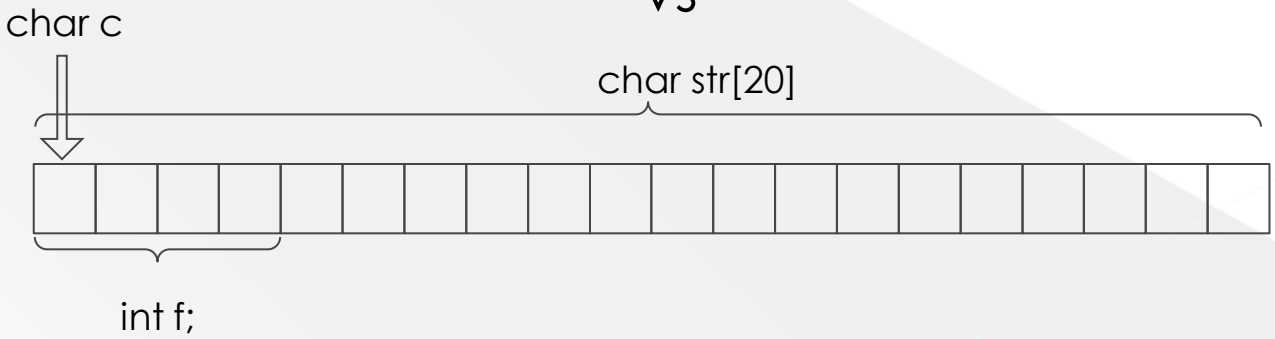
Structures vs Unions

structure patient



VS

union Data



Union : Definition (contd.)

```
#include <stdio.h>
#include <string.h>

union Data {
   char c;
   int f;
   char str[20];
};

int main( ) {

   union Data data;

   printf( "Memory size occupied by data : %d\n", sizeof(data));

   return 0;
}
```

Union : Definition

```
union [union tag] {
   member definition;
   member definition;
   ...
   member definition;
}
```

Union : Definition (contd.)

- a variable of **Data** type can store a character or an integer. It means a single variable, i.e., same memory location, can be used to store multiple types of data.

- The memory occupied by a union will be large enough to hold the largest member of the union.

Accessing Union Members

To access any member of a union, we use the member access operator (.). The member access operator is coded as a period between the union variable name and the union member that we wish to access.

Accessing Union Members

```c
#include <stdio.h>
#include <string.h>

union Data {
   char c;
   float f;
   char str[20];
};

int main( ) {

   union Data data;

   data.c = 'a';
   data.f = 220.5;
   strcpy( data.str, "C Programming");

   printf( "data.i : %d\n", data.i);
   printf( "data.f : %f\n", data.f);
   printf( "data.str : %s\n", data.str);

   return 0;
}
```

```
data.c : 1917853763
data.f : 41223605803277948604527599943 68.000000
data.str : C Programming
```

Here, the values of i and f members of union got corrupted because the final value (which is the largest - memory size wise) assigned to the variable has occupied the memory location.

Union + Structures

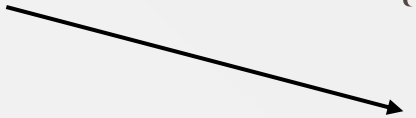
- Union and structures are user defined data types and just like any regular data type (int, float etc.) they can be used in another structure/union provided that they've been declared earlier.

Union + Structures

For example, say that, our student ID can be either 123 or PME_123, so we have to use union. Now, in struct "marks" we have to store a student's ID, we simply use the union we defined earlier here. However-

- The keyword "struct" must be written before the variable name.
- Union studentID must be declared above struct marks.

```
union studentID {
   int i;
   char str[20];
};
```



```
struct marks
{
  int subject1;
  int subject2;
  union studentID ID;
} ;
```

Check out union_example.c

Any More Problems with Library.c?

Can't Really tell which customer took which exact book. Only the info of last customer, who took that book, is stored.

Solve: We should have separate structures for book and person/lib member.

Try on your own to solve

Thank You!